Hochschule Düsseldorf
University of Applied Sciences
HSD

Advances in AI

Prof. Dr. Dennis Müller

AI Age Transformation

Nick Grabowski, 912350

https://github.com/nickno7/age_transformation

16. 02. 2025

Contents

1 Introduction

1.1 Project Description

In this project, I am focusing on AI-based age transformation with the help of StyleGAN. This involves both facial ageing and rejuvenation. For a realistic age transformation, it is important to change the age characteristics without losing the identity of the person. Using the interpretable latent space of the StyleGAN architecture, facial features such as age can be adjusted by adding a latent representation of the person with a vector that changes the age. Based on the direction of the vector, the person can be either aged or rejuvenated.
The project includes the ability to generate realistic faces using a pre-trained StyleGAN and to transform the age of these faces. This involves the use of an encoder that has learned to generate latent vectors of the StyleGAN space based on input images in order to be able to work with real people and adjust their age.

1.2 Motivation

I chose this project because I find it very exciting how quickly AI has developed and improved in terms of realistic image generation. Especially this-person-does-not-exist.com [1] fascinated me and many other people, because you could see what GANs and especially StyleGAN are capable of. It was also important to me to carry out a visual project where you can see the results directly.
The opportunity to work with generative models for the first time and to gain a deeper understanding of how they work also appealed to me.

1.3 Relevance

The question arises as to why this is needed at all and what use cases there are. AI-based age transformation is relevant and helpful in many areas. In forensics in particular, it is a suitable method for finding people who have been missing for many years by creating images that show what the person might look like today. This is particularly helpful with children, as facial structures and appearance change considerably in just a few years.
Another use case is the entertainment industry. Age transformations are used to create realistic past or future versions of actors without having to use other actors.
In addition, age transformations can be used to reconstruct historical persons in order to show how deceased persons from the past would look today.

2 State of the Art

The ageing of faces using artificial intelligence has made considerable progress in recent years. A central point is the use of Generative Adversial Networks (GANs), which make it possible to generate and manipulate realistic images [2].
A significant milestone was the development of the StyleGAN, which offers high quality and control in image synthesis [3]. Research has shown that the latent space of GANs contains semantic information that enables targeted modification of attributes such as age, gender or facial expression
A few years after the publication of GANs, the latent space and its interpretability were researched. This involved investigating how various attributes, such as age, gender or facial expression, are represented in the latent space of GANs. It was found that specific features of a face could be changed by targeted modification of the latent vectors, which enables precise control of age [4].
Another method for facial modification is CycleGAN, which can translate unsupervised between two domains, for example between young and old faces. This enables the aging or rejuvenation of faces without pairwise training data [5].
Die Invertierbarkeit von GANs ist ein weiteres relevantes Forschungsgebiet. Es untersucht, wie man von einem gegebenen Bild auf den entsprechenden latenten Code schließen kann, um gezielte Bildmanipulationen durchzuführen. Studien haben gezeigt, dass es möglich ist, den latenten Code effizient zu rekonstruieren [6].
The invertibility of GANs is another relevant area of research. It investigates how to infer the corresponding latent code from a given image in order to perform targeted image manipulations. Studies have shown that it is possible to efficiently reconstruct the latent code [6].
Despite this progress, there are still challenges, such as the problem of mode collapse, where certain features or objects are not generated correctly. Research has shown that GANs tend to neglect certain object classes, highlighting the need for further research to improve model generalization [7].

3 Methodology

The project is divided into different chapters, as many building blocks were required to realize the goal. I worked with the PyTorch environment to program the StyleGAN architecture as well as the encoder and the latent optimizer. Due to time constraints, I used a pre-trained mesh for the StyleGAN. This gave me the opportunity to generate artificial faces with random z vectors and to use them for the further course, e.g. for training the encoder.

3.1 StyleGAN-Implementation

3.1.1 Introduction

This implementation is based on the StyleGAN architecture, which combines a progressive growth strategy for generator networks with style-based controls. The architecture includes several essential building blocks, including:

3.1.2 Custom Layers

- **MyLinear**: A fully connected layer with uniform learning rate scaling.
- **MyConv2d**: A convolutional layer with upscaling option.
- **NoiseLayer**: Fügt Rauschen für Variation in den Features hinzu.
- **StyleMod**: Adds noise for variation in the features.
- **PixelNormLayer**: Standardization of features at pixel level.
- **BlurLayer**: Implementation of a low-pass filter to stabilize the training.
- **Upscale2d**: Upscaling function for images.

3.1.3 Mapping Network

The Mapping-Network ($G_{mapping}$) processes the latent space z through multiple fully connected layers and projects it into **W-space** to provide fine-grained control over the generated image.

3.1.4 Synthesis Network

The $G_{synthesis}$ network converts the latent vector w step by step into a high-resolution image. It consists of:

- **InputBlock**: Initializes the feature grid (4×4 pixels) either with a learned constant or by transforming the latent code.
- **GSynthesisBlock**: Continuous blocks with upscaling and convolution.

The network grows progressively from low to high resolution, creating fine details in the images.

3.1.5 Truncation Trick

An optional **Truncation layer** interpolates the latent vector in the direction of the average vector to control the diversity of the generated images.

3.1.6 Conclusion

This implementation provides a solid foundation for StyleGAN models and can be further enhanced by techniques such as progressive growth strategies and optimized normalization approaches.

3.2 Encoder

The following module implements an encoder network that projects images into the latent style space (W+) of a pre-trained StyleGAN model. It also provides functions for generating a synthetic dataset and for training and evaluating the encoder.
I perform the age transformation using a manipulation of the StyleGAN's latent vector. Therefore, the encoder is designed for the task of projecting an input image that was not generated by the StyleGAN into the latent space of the StyleGAN in order to subsequently manipulate the vector and change the age.

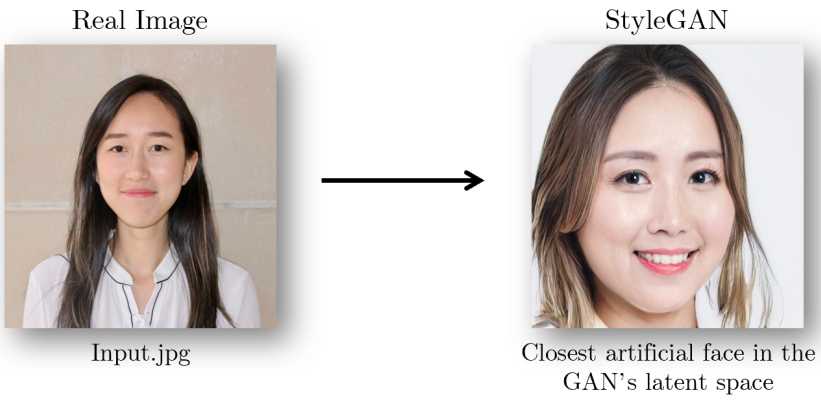

Figure 1 Source: `https://github.com/sshh12/Inverse-Style-GAN`

3.2.1 Dataset

For the encoder, I created a custom dataset with 50,000 image, latent vector pairs using the StyleGAN implementation. The following steps were carried out to generate the data set:

1. Random generation of z-vectors with 512 dimensions.

2. z-vector passes through the StyleGAN mapping network to obtain the latent w-vector, which contains valuable information such as the age structure.
3. Generation of the corresponding images with the StyleGAN synthesis model.
4. Storage of the images and the corresponding latent vectors.

This data set was necessary to train an encoder network that learns from the faces (input) to generate corresponding latent vectors that represent the face as accurately as possible.

```
def generate_dataset(device, g_mapping, g_synthesis, num_samples=50000):
    for i in range(num_samples):
        z_sample = torch.randn(1, 512).to(device)  # Zufälliger z-Vektor
        latent = g_mapping(z_sample)  # Mapping zu w-Vektor
        with torch.no_grad():
            image = g_synthesis(latent)  # Gesicht generieren
        save_image(image.cpu(), f"images/{i:05d}.png")
        torch.save(latent.cpu(), f"latents/{i:05d}.pt")
```

“Abridged/simplified version”

3.2.2 Architecture

The encoder is based on a pre-trained ResNet-50 model, which is used to extract deep image features. A mapping to the StyleGAN latent style space is then performed. The architecture includes:

- A modified ResNet-50 architecture for feature extraction.
- An additional convolution layer to reduce the feature dimension.
- Fully Connected Layers to project the image features into the latent space.

```
class Encoder(nn.Module):
    def __init__(self, latent_dim=512, w_plus_layers=18):
        super(Encoder, self).__init__()
        self.resnet = resnet50(pretrained=True)
        self.resnet = nn.Sequential(*list(self.resnet.children())[:-2])
        self.conv = nn.Sequential(
            nn.Conv2d(2048, latent_dim, kernel_size=1, stride=1),
            nn.BatchNorm2d(latent_dim),
            nn.LeakyReLU(0.2)
```

```
        )
        self.flatten = nn.Flatten()
        self.fc1 = nn.Sequential(
            nn.Linear(latent_dim * 7 * 7, 1024),
            nn.Dropout(0.5),
            nn.LeakyReLU(0.2)
        )
        self.fc2 = nn.Linear(1024, latent_dim * w_plus_layers)

    def forward(self, x):
        x = self.resnet(x)
        x = self.conv(x)
        x = self.flatten(x)
        x = self.fc1(x)
        x = self.fc2(x)
        x = x.view(-1, 18, 512)
        return x
```

Training process

The encoder model is trained with a generated data set. The training process includes:

- Using a combination of dataset transformations, such as resizing, random cropping and normalization.
- Using the `logcosh` loss function to measure the difference between the predicted and true latent vectors.
- Using Adam as an optimization algorithm with a learning rate scheduler.
- Save and load checkpoints to continue training in case of interruptions.

```
optimizer = optim.Adam(encoder.parameters(), lr=0.1, weight_decay=1e-5)
scheduler = torch.optim.lr_scheduler.ReduceLROnPlateau(optimizer,
                mode='min', factor=0.5, patience=2, min_lr=1e-6)

for epoch in range(start_epoch, epochs):
    encoder.train()
    epoch_loss = 0
    for imgs, true_latents in train_loader:
        imgs, true_latents = imgs.to(device), true_latents.to(device)
```

```
        predicted_latents = encoder(imgs)
        loss = logcosh_loss(true_latents, predicted_latents)
        optimizer.zero_grad()
        loss.backward()
        optimizer.step()
```

Conclusion

This module provides a complete pipeline system for encoding images into StyleGAN's latent style space. It includes the dataset creation as well as the training and evaluation processes of the encoder.

3.3 Latent Optimizer

Since the encoder only provides an approximation of the real input image and the output is usually not similar enough to it, another method was necessary. I implemented the Latent Optimizer for this purpose. This is used to obtain the most precise latent embedding possible that reconstructs the input image as accurately as possible. This is particularly important in order to be able to carry out a realistic age transformation without losing the identity.
In order to minimize the difference between the reconstructed image (output of the encoder) and the real image, an optimization process is used to gradually refine the latent vector.

3.3.1 Procedure

The process takes the latent vector generated by the encoder as the starting point for optimization. A ResNet model is used to extract the perceptual features of the image and compare the real and generated image.
In addition, the MSE loss is used to reduce the pixel difference between the two images.
The total loss is the sum of the perceptual and MSE loss, resulting in the following loss function.

$$\mathbf{loss} = 0.6 \cdot \mathbf{perceptual\_loss} + 0.4 \cdot \mathbf{mse}$$

```
def optimize_latent(image, device, encoder, g_synthesis):
    """Optimizes latent representation of an input image."""
    upsample = torch.nn.Upsample(scale_factor = 224/1024,
                                 mode = 'bilinear')
    img_p = upsample(image.clone())
```

```
    perceptual = ResNet_perceptual().to(device)

    # MSE loss object
    mse_loss = nn.MSELoss()
    # send image through encoder for first approximation
    latent = encoder(image)
    # Optimize latent code in each backward step
    optimizer = optim.Adam({latent},lr=0.01,betas=(0.9,0.999),eps=1e-8)

    loss_ = []

    # latent optimization loop
    for i in range(2000):
        optimizer.zero_grad()
        syn_img = g_synthesis(latent)
        syn_img = (syn_img + 1.0)/2.0

        # calculate losses
        mse, per_loss = loss_function(syn_img, image, img_p, mse_loss,
                                      upsample, perceptual)

        loss = 0.6 * per_loss + 0.4 * mse
        loss.backward()
        optimizer.step()

        loss_np = loss.detach().cpu().numpy()
        loss_mse = mse.detach().cpu().numpy()
        loss_per = per_loss.detach().cpu().numpy()
        loss_.append(loss_np)

        # print loss and save image every 500th iteration
        if (i + 1) % 500 == 0:
            print(f"Iteration{i+1}: loss -- {loss_np},
                  mse_loss -- {loss_mse},  percep_loss -- {loss_per}")
            save_image(syn_img.clamp(0,1),
                       f"outputs/face_{i+1}_iterations.png")

    return latent
```

3.4 Age Transformation

The latent space of the StyleGAN is interpretable and the latent vectors contain a lot of information about facial features [4].
This property can be exploited to manipulate and transform faces in a variety of ways. Among other things, age can also be adjusted by selectively manipulating the latent vector. Linear separation methods were used to identify which directions exist in the latent space that correlate with certain facial features.
I made use of these directions/boundaries by using the boundary that describes the age of the person in the latent space for the age transformation. The age transformation then takes place by shifting the latent vector in the direction of the boundary. Depending on the direction in which the latent vector is shifted, the face is either rejuvenated or aged.
The new latent vector is calculated accordingly as follows

$$\mathbf{w}_{\text{aged}} = \mathbf{w} + \alpha \cdot \mathbf{b}$$

The factor alpha α determines how strong the ageing should be.
If a negative value is selected, the face is rejuvenated. With a positive value, on the other hand, the face is aged.
This new latent vector $\mathbf{w}_{\text{aged}}$ can then be entered into the $G_{synthesis}$ network to obtain the corresponding aged image.

3.4.1 Interpolation

To illustrate the ageing process, an interpolation was carried out between the original latent vector ω and the aged vector $\mathbf{w}_{\text{aged}}$. A new latent vector is calculated step by step:

$$w_{\text{interp}} = (1 - a) \cdot w + a \cdot w_{\text{aged}}$$

with α as interpolation parameter in the range [0,1].
This interpolation process enables a smooth transition animation between the original and the aged face. In each step, an image is generated and saved from the interpolated latent vector using the StyleGAN synthesis model. The individual images are then merged into a GIF animation to visually represent the aging process.

```
num_steps = 75

# Generate interpolated images
interpolated_images = []
with torch.no_grad():
    for a in np.linspace(0, 1, num_steps):
```

```
        z = ((1 - a) * latent) + (a * aged_w)
        result = g_synthesis(z)
        result = (result.clamp(-1, 1) + 1) / 2.0  # Normalize
        result = result.cpu()
        interpolated_images.append(result)
```

4 Results

The age transformation worked very well with generated faces and simulated extremely realistic aging. The results can be found in the README of the GitHub repository. It can be seen that both aging and rejuvenation work and produce good transformations.
The age transformation with real faces achieved poorer results. Various factors played a role in this, which I explain in more detail in the next section “Challenges”.
The main reason for the poorer results is the complex transformation of the input image into the latent space of the StyleGAN. The problem is that the encoder cannot reconstruct the face exactly and you only have an approximation of the face. Latent optimization was used for this, but this often leads to artifacts and blurred faces due to the MSE loss. [8]
The resulting latent vectors are useless for the age transformation, as the artifacts also occur in the manipulated/aged faces.

5 Challenges

Some difficulties arose during the course of the project. In the beginning, I only worked with latent optimization and without encoder architecture. The results could be improved, as artifacts often appeared in the image. As a result, the face was distorted and could not be used for the age transformation in the further course. The image quality of the generated faces was generally a limiting factor, as only high-resolution faces could be subsequently manipulated. Blurred and distorted images were also blurred after the age transformation, so that unfortunately no good and realistic result was achieved.

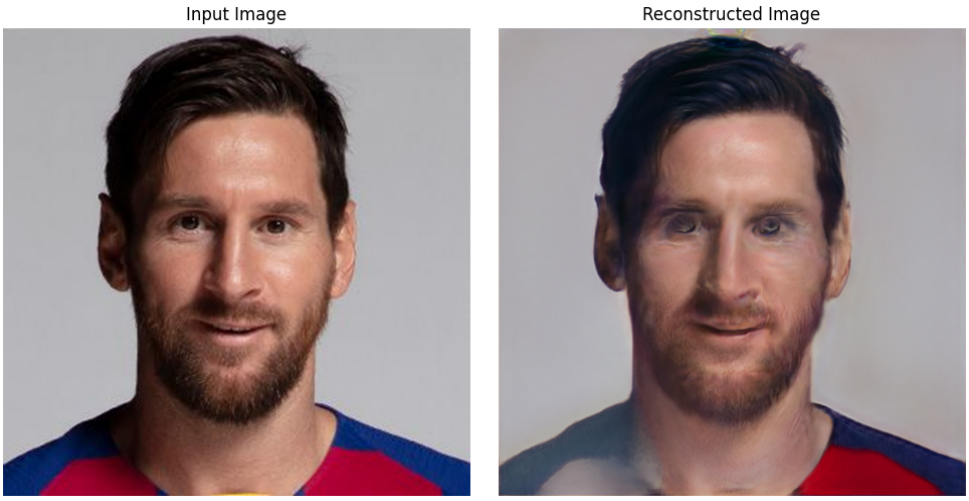

Figure 2 Unzureichende Bildqualität nach Latent Optimierung

Even after implementing the encoder, the faces reconstructed from the latent vectors did not come close enough to the input images. The age transformation with input images is therefore only possible in a few cases. The bottleneck of the encoder was the generated training data set. The generated faces, with their corresponding latent vectors, were always in a similar position with almost identical ratios. This is due to the generation of faces with the StyleGAN architecture. When input images with different positioning, different lighting conditions or other features were converted to a latent vector, the encoder had difficulty finding a meaningful latent representation.

6 Next Steps

To optimize the entire pipeline, I would take the following steps. Firstly, the encoder architecture needs to be improved to get better approximations of the input image. This could involve creating a larger data set and providing more variation so that the encoder can cope with varying inputs and output meaningful latent vectors. The next step would also be to improve the optimization process of this latent vector so that blurred images and artifacts no longer occur. If this is ensured, the age transformation will also output very realistic results with real input images.

- Improve latent optimization
  - Use adversarial training methods, e.g. by integrating an additional discriminator, to further improve the quality of the reconstructed faces.

  Extend

- data set
  - Create a larger data set and provide more variation so that the encoder can also cope with varying inputs and output meaningful latent vectors.

- Evaluation of the age transformation
  - Objective metrics, such as FID score, to measure progress.
  - User study with people who subjectively evaluate the age transformation with their face.

Appendix

Literaturverzeichnis

Bibliography

[1] This Person Does Not Exist. *This Person Does Not Exist*. Zugriff am 05. Februar 2025. 2025. URL: https://this-person-does-not-exist.com.

[2] Ian J. Goodfellow et al. "Generative Adversarial Networks". In: *arXiv preprint* 1406.2661 (2014). DOI: 10.48550/arXiv.1406.2661. arXiv: 1406.2661 [stat.ML]. URL: https://arxiv.org/abs/1406.2661.

[3] Tero Karras, Samuli Laine, and Timo Aila. "A Style-Based Generator Architecture for Generative Adversarial Networks". In: *arXiv preprint* arXiv:1812.04948 (2018). URL: https://arxiv.org/abs/1812.04948.

[4] Yujun Shen et al. "Interpreting the Latent Space of GANs for Semantic Face Editing". In: *arXiv preprint* (2019). arXiv: 1907.10786. URL: https://arxiv.org/abs/1907.10786.

[5] Jun-Yan Zhu et al. "Unpaired Image-to-Image Translation using Cycle-Consistent Adversarial Networks". In: *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*. 2017, pp. 2223–2232. DOI: 10.1109/ICCV.2017.244.

[6] Weihao Xia et al. "GAN Inversion: A Survey". In: *arXiv preprint* arXiv:2101.05278 (2021). URL: https://arxiv.org/abs/2101.05278.

[7] David Bau et al. "Seeing What a GAN Cannot Generate". In: *arXiv preprint* arXiv:1910.11626 (2019). URL: https://arxiv.org/abs/1910.11626.

[8] Grigory Antipov, Moez Baccouche, and Jean-Luc Dugelay. "Face Aging with Conditional Generative Adversarial Networks". In: *arXiv preprint* (2017). arXiv: 1702.01983. URL: https://arxiv.org/pdf/1702.01983.

Abbildungen